

ARCHITECTURE REFERENCE

01 Overview & roles

02 Data model

03 Auth & access

04 Benefit Wallet

05 Leave & WFH

06 Capacity Planner

07 Email & audit

08 Route map

09 Known gaps

10 SDLC flowcharts

Next.js App Router

Drizzle

Supabase

Resend

Vercel Cron

INTERNAL REFERENCE · SS-BENEFITS-AND-LEAVE-MANAGER

Benefit Wallet + Leave/WFH Manager – System Architecture & Flow Reference

A working map of every flow shipped to date — who can do what, which tables depend on which, and how a request moves from submission to audit trail. Verified directly against the current source tree, not against prior docs.

Generated **2026-07-03** Covers epics **KAN-4x – KAN-81** Branch **dev**

01 – OVERVIEW

Two modules, one identity model

The app is an internal employee portal with two functional modules sharing a single auth and role model: the **Benefit Wallet** (annual category-capped expense allowances) and the **Leave & WFH Manager** (apply + multi-level approval). A third, newer slice — the **Capacity Planner** — sits on top of leave data to warn managers about understaffing before it happens.

Five roles: `employee`, `team_lead`, `project_manager`, `hr_head`, `admin`.

Reporting lines are never hard-coded — they're data on the `users` row

(`teamLeadId`, `projectManagerId`, both self-referencing foreign keys back to `users.id`), so the same code path serves any org shape.

Enforcement is always layered: a page-level `requireAccess(path)` redirect, plus every Server Action independently re-checking `assertCan()` and ownership. The route guard is never trusted alone.

02 – DATA MODEL

13 tables in four clusters

Everything hangs off `users`. There is no separate "team" table — a team is just the set of rows whose `teamLeadId` or `projectManagerId` points at a given manager.

CLUSTER A – IDENTITY & ORG

users

- `id`
- `name / email` (unique)
- `role`
- `teamLeadId` → `users.id`
- `projectManagerId` → `users.id`
- `department`
- `isCriticalRole`
- `joinDate / createdAt`

CLUSTER B – BENEFIT WALLET

benefit_categories

- `id`
- `name`

benefit_claims

- `id`
- `userId` → `users`

annualCapPaise
fyStart
carryover

categoryId → benefit_categories
amountPaise / expenseDate / vendor
documentUrl / documentHash
status
verificationResult jsonb
approverId → users
decisionReason / fy

CLUSTER C – LEAVE & APPROVALS

leave_types

● id
code (unique)
accrualPerMonthDays
openingBalanceDays
maxBalanceDays
carryForward
deductsBalance

leave_balances

● id
userId → users
leaveTypeId → leave_types
balanceDays
fy

leave_requests

● id
userId → users
kind (leave/wfh)
leaveTypeId → (null
leave_types =
WFH)
fromDate / toDate / halfDay
/ workingDays
status / currentLevel
teamLeadId → users
projectManagerId → users

approvals

● id
requestId → leave_requests
level
approverId → users
decision / reason

approval_policy (singleton)

● id = "default"
routingMode
wfhAutoApproveMaxDays
ccEmails jsonb
updatedBy → users

CLUSTER D – CAPACITY & OPS

staffing_threshold

● id
scope (org/department)
scopeValue
minAvailablePercent
updatedBy → users

team_capacity_snapshot

● id
date
scopeType (team/dept/org)
scopeId (loose text ref)
totalHeadcount /
availableCount
capacityPercent

holidays

● id
date
name
location

email_log

● id
toAddress
subject
template
status

audit_log

● id
actorId → users
action / entity / entityId
payload jsonb

Foreign-key map

```
users.teamLeadId → users.id (self)
users.projectManagerId → users.id (self)
benefit_claims.userId → users.id
benefit_claims.categoryId → benefit_categories.id
benefit_claims.approverId → users.id
leave_balances.userId → users.id
leave_balances.leaveTypeId → leave_types.id
leave_requests.userId → users.id
leave_requests.leaveTypeId → leave_types.id
leave_requests.teamLeadId → users.id (snapshot)
leave_requests.projectManagerId → users.id (snapshot)
approvals.requestId → leave_requests.id
approvals.approverId → users.id
approval_policy.updatedBy → users.id
staffing_threshold.updatedBy → users.id
audit_log.actorId → users.id
```

`team_capacity_snapshot` and `email_log` deliberately carry **no hard FK** —
`scopeId` and `entityId` are loose text/uuid references, since the daily cron job
writes on behalf of scopes (departments, ad-hoc teams) that aren't first-class rows.

Enumerated types

role employee team_lead project_manager hr_head admin

claim_status

draft submitted auto_approved pending_hr approved rejected reimbursed

leave_request_status applied pending_l1 pending_l2 approved rejected cancelled

request_kind leave wfh

decision approved rejected

routing_mode sequential parallel

staffing_threshold_scope org department

Session bootstrap → capability check → ownership check

LOGIN → FIRST PAGE RENDER

SRC/SERVER/AUTH/CURRENT-USER.TS

1

Supabase session

Client authenticates; server reads the session on every request.

→

2

Find or create**users row**

First login only. Role comes from `user_metadata.app_role`, whitelisted via `resolveSignupRole`.

→

3

ensureLeaveBalances

Seeds opening balances per leave type for the current FY if missing.

→

4

requireAccess(path)

Redirects to `/login` (no session) or `homeRouteFor(role)` (wrong module).

`SIGNUP_ROLES` intentionally excludes `hr_head` and `admin` — a client can never self-elevate past `employee` / `team_lead` / `project_manager` at signup.

`MODULE_ACCESS` (`src/server/users.ts`) is the single source of truth consumed by three different call sites, so nav visibility and enforcement can never drift apart:

- `canAccessPath(role, path)` — prefix match, used by the server-side route guard.
- `homeRouteFor(role)` — manager roles land on `/approvals`, HR roles on `/expenses`, everyone else on `/dashboard`.
- `NAV_SECTIONS` — same prefixes drive what the sidebar renders.

Capability matrix (`src/server/auth/rbac.ts`)

CAPABILITY	EMPLOYEE	TEAM LEAD	PROJECT MGR	HR HEAD	ADMIN
submitExpense / applyLeave	✓	✓	✓	✓	✓
approveLeaveL1		✓			

CAPABILITY	EMPLOYEE	TEAM LEAD	PROJECT MGR	HR HEAD	ADMIN
approveLeaveL2			✓		
approveExpense				✓	
exportAvailability		✓	✓	✓	✓
viewDepartmentOverview				✓	✓
configurePolicy (thresholds, approval policy, admin CRUD)				✓	✓
runReimbursementExport				✓	
manageUsers (reporting lines, role changes)					✓

Ownership is layered on top of capability: `assertOwnership()` always allows a user to act on their own record; beyond that, only `hr_head` / `admin` may act on someone else's.

04 – BENEFIT WALLET

Submit → verify → approve → reimburse

Category caps (Sports ₹15,000, Learning ₹45,000/FY) are enforced against a **derived** balance — nothing is decremented on submit. `getCategoryBalances` (`src/server/employee/balances.ts`) recomputes `approvedPaise` and `pendingPaise` live from `benefit_claims.status` every time it's read.

SUBMITEXPENSEACTION

SRC/SERVER/ACTIONS/EXPENSE.TS

1

Validate + auth

Zod schema;
`assertCan(role, "submitExpense")`
always own claim.

2

Upload receipt

Private Supabase bucket,
path keyed by SHA-256
hash — never a public
URL.

3

Claude OCR parse

`claude-haiku-4-5-20251001` extracts
amount/date/vendor/confidence;
any failure degrades to zero-
confidence.

4

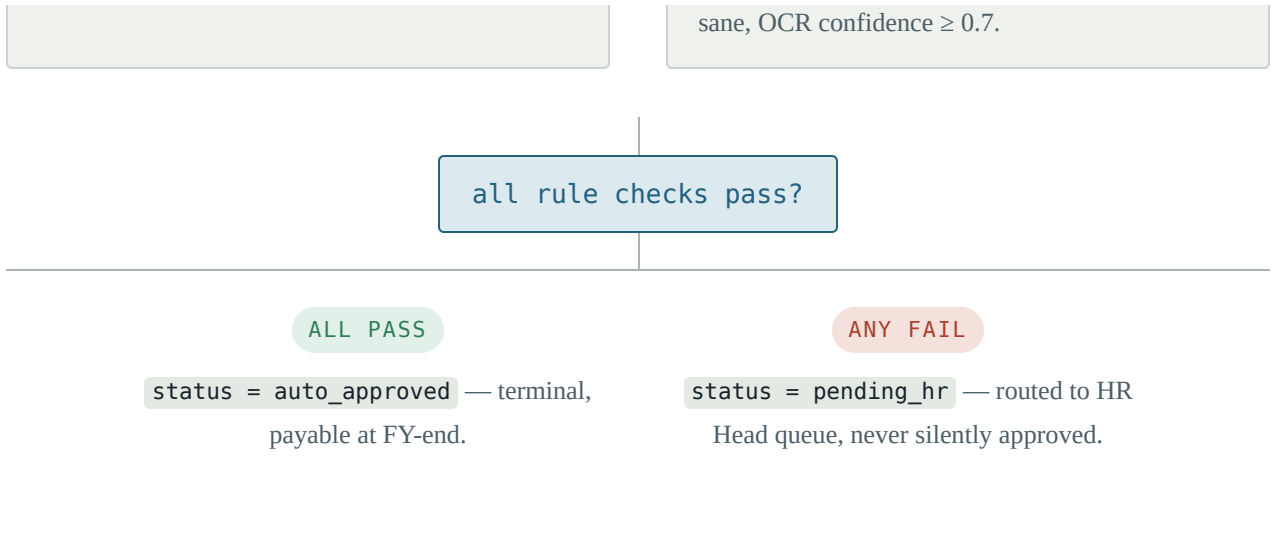
Duplicate check

By document hash, or category+amount+date
match.

5

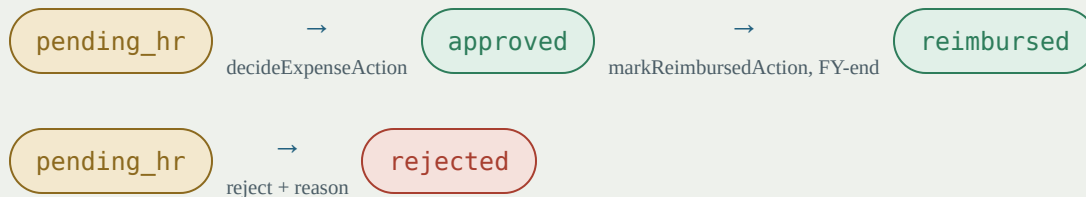
Rule engine

`runRuleChecks()` — file present, amount
matches OCR, within FY, within balance, vendor



Hard rule enforced in code: an inconclusive verification result can never resolve to `auto_approved` — the ternary in `expense.ts` only takes that branch when `result.passed` is strictly true.

HR decision → reimbursement

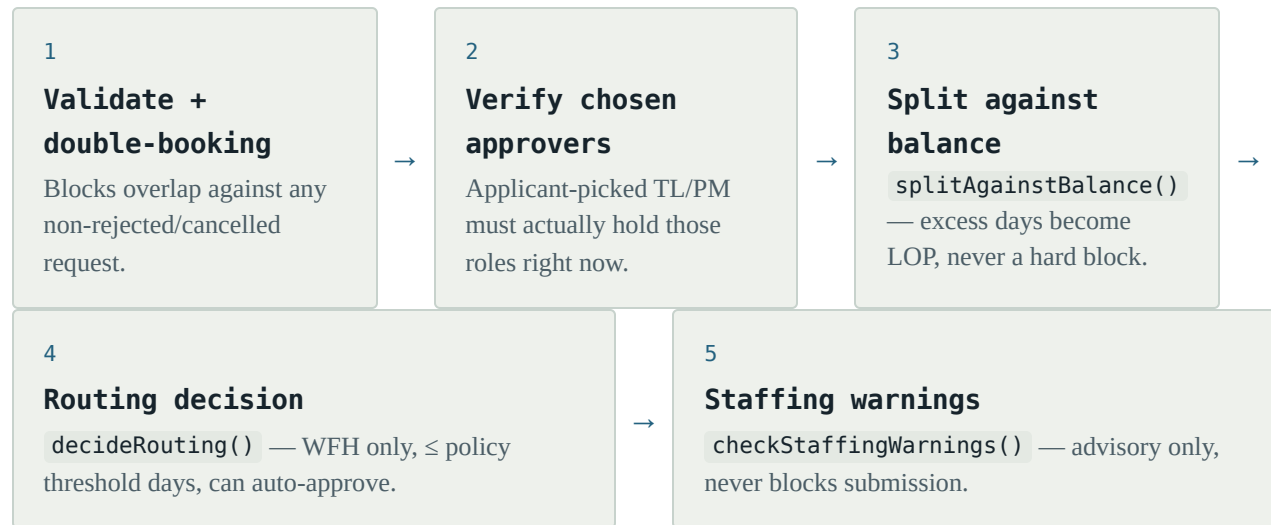


`decideExpenseAction` (HR Head only) requires a reason on rejection, writes `audit_log`, and best-effort emails the applicant via the `expense_decision` template. `markReimbursedAction` batches every `auto_approved` / `approved` claim for a financial year, flips them to `reimbursed` in one transaction, and writes a single `audit_log` row (`reimburse_batch`) with the claim IDs, total, and employee count — it records the payout decision, it does not move money.

Receipts are only ever exposed through `getReceiptUrlForClaim`: a 60-second signed URL, gated by ownership (own claim, or HR Head/Admin), and every disclosure writes an `audit_log` row (`view_receipt`).

APPLYLEAVEACTION

SRC/SERVER/ACTIONS/LEAVE.TS



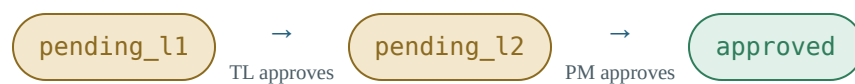
Hard rule enforced in code: balance-deducting leave can never auto-approve, only WFH can — the routing check gates on `kind === "wfh"` before it even looks at the day threshold.

Approval routing – sequential vs. parallel

Routing mode is a single global setting (`approval_policy.routingMode` , HR-configurable at `/settings/approvals`).

SEQUENTIAL

TEAM LEAD → PROJECT MANAGER



A reject at either level jumps straight to `rejected` . The PM is only notified by email once L1 approves.

PARALLEL

TEAM LEAD OR PROJECT MANAGER, EITHER CAN CLOSE IT



Both TL and PM are notified up front at apply time; whichever decides first resolves the request — the row never enters `pending_l2` .

`decideLeaveAction` re-checks ownership every time
(`row.teamLeadId / row.projectManagerId` must equal the acting user, else
`ForbiddenError: "Not your direct report."`) and re-runs the KAN-77
staffing warnings so the approver sees current impact, not what it looked like at apply
time.

Balance deduction – the audited step

Balance is deducted **only on final approval** of a balance-deducting leave type —
never at submission, never speculatively. The deduction and the `audit_log` row
(`deduct_leave_balance`) are written inside the same transaction as the approval.
Cancellation before final approval therefore needs no restoration — nothing was
ever deducted.

Accrual (`src/server/leave/accrual.ts`) tops a balance up to the KEKA-style
expected-accrued value and never claws back used days; every application writes
`audit_log ( accrue_leave_balance )`. **It is not wired to any cron** in this repo —
see §09.

06 – CAPACITY PLANNER · KAN-73 → KAN-81

Two independent paths read the same leave data

The Capacity Planner has no dedicated "capacity" table of record for planning — it
derives everything from `leave_requests` through one shared function,
`getAvailabilityForRange()`. What differs is *when* that function runs and *what*
happens to the result:

PATH A – DAILY CRON → HISTORICAL SNAPSHOT → ALERT

RUNS ONCE/DAY, UNATTENDED

1

Vercel Cron

18:30 UTC daily,
`vercel.json` → `GET`
`/api/cron/capacity-`
`snapshot`.

→

2

CRON_SECRET auth

Bearer token only — no
user session, this route has
no RBAC.

→

3

Per-scope write

Org (1) + every department
+ every manager's team, via
`Promise.allSettled` —
one failure doesn't sink the
rest.

→

4

5

team_capacity_snapshot

Approved-only figure for that day, one row per scope.

checkLowStaffingAndNotify

Org/department scopes only — never "team".

capacityPercent < threshold?

BREACH

Dedup-check `email_log` by exact subject (`Low staffing alert: {scope} on {date}`) + template, then email every `hr_head` (CC'd with the department's TL/PM on a department breach).

HEALTHY

No-op. Threshold resolution: department override wins, else org default.

PATH B — LIVE 2–4 WEEK FORECAST

COMPUTED ON EVERY PAGE LOAD, /AVAILABILITY

1

resolveTeamScope

TL/PM forced to own reports; HR/Admin may pick any team.

→

2

getAvailabilityForRange

21-day rolling window, computed live from `leave_requests`.

→

3

buildForecastPoints

Confirmed (approved-only) vs. at-risk (approved+pending) series per day.

This is the dependency to remember: the live forecast on `/availability` *never reads* `team_capacity_snapshot` — it recomputes from scratch every render. The snapshot table is a pure write sink today; the historical-trend UI that would read it back doesn't exist yet (see [§09](#)). The two paths will only ever agree on the approved-only number for days already in the past.

Availability filters & CSV export KAN-80

`getAvailabilityForRange` takes an optional `leaveTypeId` filter — narrowing to one leave type also excludes WFH rows (they carry a null `leaveTypeId`). The `/availability` page validates every query param (`role` , `leaveType` , `from` , `to`) against an enum/UUID/ISO-date shape before use; anything malformed is dropped, not passed through.

CSV export is a real Route Handler, not a Server Action — `POST /api/availability/export` — so the response is a downloadable file. It re-derives scope through the same `resolveTeamScope` /ownership rules as the page, gated by `exportAvailability` (+ `viewDepartmentOverview` specifically for a department-scope export), then pipes the same `getAvailabilityForRange` result into a pure, DB-free `buildAvailabilityCsv()`.

07 – EMAIL & AUDIT TRAIL

Two logs, two different jobs

`email_log` is a best-effort delivery record — every send is wrapped so a logging failure can never break the action that triggered it — **except** in one place: the low-staffing alert reads it back to decide whether it already sent today's alert for a scope. `audit_log` is the compliance trail for state changes that must never happen silently (a hard project rule).

EMAIL TEMPLATE	TRIGGER	RECIPIENTS
<code>leave_auto_approved</code>	WFH auto-approved at apply time	Applicant
<code>leave_l1_request</code>	New leave/WFH submitted	Team Lead
<code>leave_l2_request</code>	Parallel: at apply. Sequential: when L1 approves	Project Manager
<code>leave_decision</code>	TL or PM approves/rejects	Applicant
<code>expense_decision</code>	HR Head approves/rejects a claim	Applicant
<code>low_staffing_alert</code>	Daily snapshot below threshold (org/dept only)	All HR Heads (+ dept TL/PM as CC)

Audit-logged actions (non-exhaustive, the ones the hard rules protect)

- `deduct_leave_balance` — written in the same transaction as the balance mutation.
- `view_receipt` — written on every signed-URL issuance, not just the first.
- `reimburse_batch` — claim IDs, total paise, employee count.
- `submit_expense`, `approve_expense` / `reject_expense`, `apply_leave` / `auto_approve_leave`, `approve_leave_l{n}` / `reject_leave_l{n}`,

`cancel_leave`, `accrue_leave_balance`, `staffing_threshold.upsert` — one row per admin/config mutation too.

08 — ROUTE MAP

Every screen, its allowed roles, its data source

ROUTE	ROLES	BACKED BY
<code>/dashboard</code>	all	<code>getDashboardData</code> , <code>getCategoryBalances</code> — real DB data
<code>/submit</code>	all	<code>submitExpenseAction</code> , my-claims list
<code>/leave</code>	all	<code>applyLeaveAction</code> , my-requests, cancel
<code>/approvals</code>	Team Lead, Project Manager	<code>getApprovalQueue</code> , <code>decideLeaveAction</code>
<code>/calendar</code>	TL, PM, HR Head, Admin	<code>getTeamCalendar</code> — events, not %
<code>/availability</code>	TL, PM, HR Head, Admin	heatmap + forecast + CSV export
<code>/expenses</code>	HR Head, Admin	<code>getHrExpenseQueue</code> , <code>decideExpenseAction</code>
<code>/expenses/history</code>	HR Head, Admin	<code>getDecidedClaims</code>
<code>/expenses/export</code>	HR Head, Admin	reimbursement batch + <code>markReimbursedAction</code>
<code>/reports</code>	HR Head, Admin	spend, approval-rate, leave, queue aggregates
<code>/departments</code>	HR Head, Admin	<code>getDepartmentOverview</code> , drills into <code>/availability</code>
<code>/admin</code>	HR Head, Admin (<i>user mgmt: Admin only</i>)	users, categories, leave types, holidays CRUD
<code>/settings/approvals</code>	HR Head, Admin	<code>approval_policy</code> config
<code>/settings/staffing-thresholds</code>	HR Head, Admin	<code>staffing_threshold</code> config

Two API routes sit outside the page tree: `POST /api/availability/export` (session-authenticated, RBAC-checked) and `GET /api/cron/capacity-snapshot` (bearer-token-authenticated, no user session at all — by design, since Vercel Cron has no user to act as).

09 — KNOWN GAPS

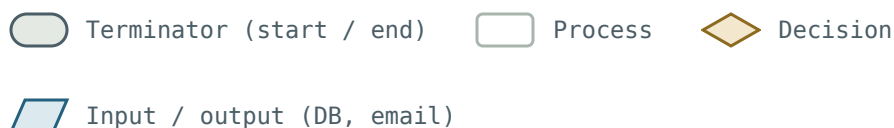
What's built but not yet load-bearing

- `team_capacity_snapshot` has been written to daily since KAN-79 shipped, but no screen reads it back yet — the historical-trend view is a future story, not a bug.
- Leave accrual (`runAccrualForUser`) and FY carry-forward (`carryForwardBalance`) are fully implemented and unit-tested but not invoked by any scheduled job — unlike the capacity snapshot, there is no cron wired to run them.
- `claim_status.draft` and `leave_request_status.applied` exist in the enums but no current code path produces them — both flows move straight to their next real state on submit.
- The auth rate limiter (`TokenBucketLimiter`) is in-memory and per-instance — it will not hold a durable limit across multiple serverless instances without a shared store (e.g. Redis).

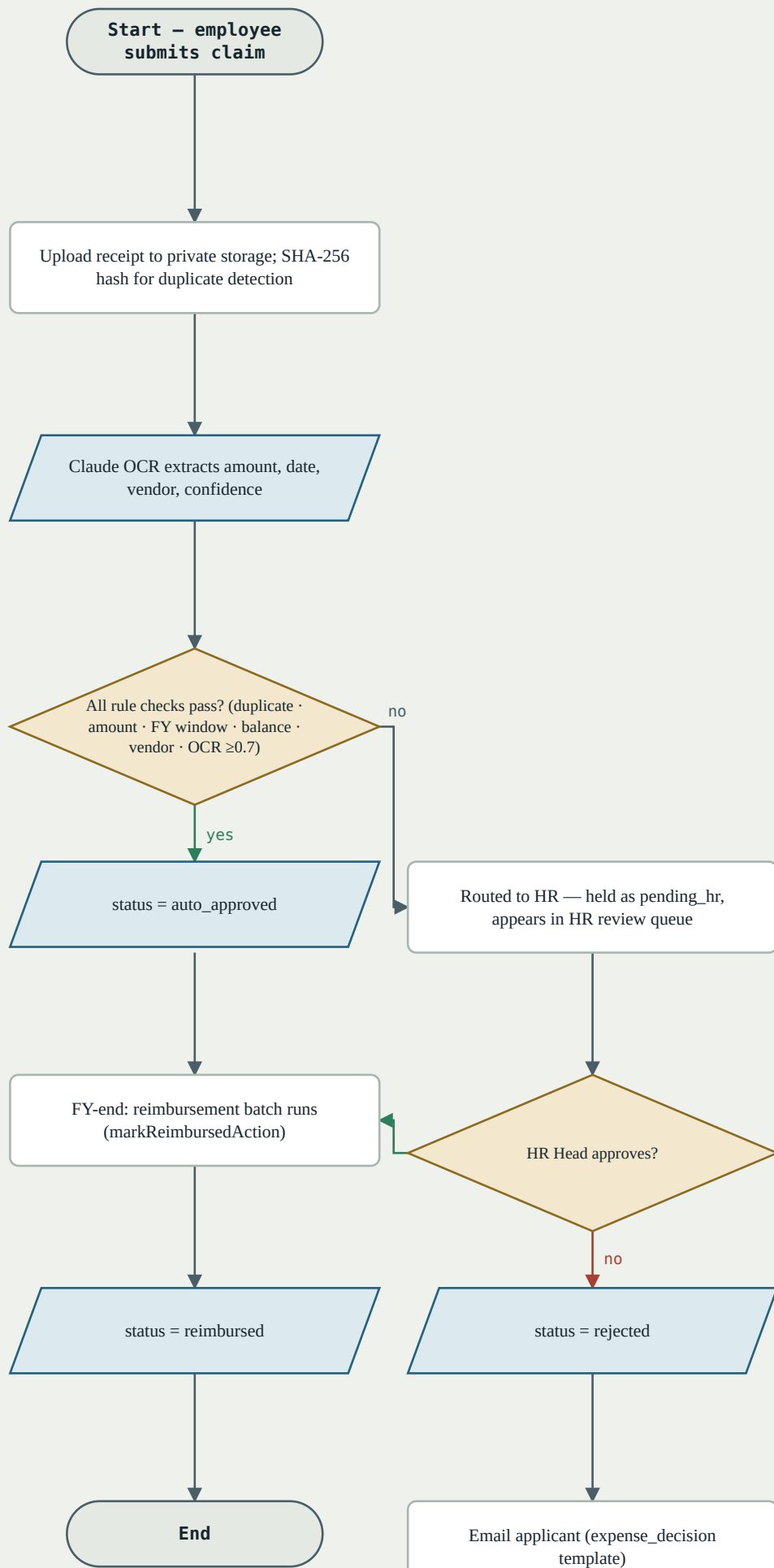
10 – SDLC FLOWCHARTS

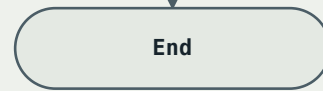
The same three flows, in standard flowchart notation

Terminator / process / decision / input-output — the four ANSI flowchart shapes — redrawn for the three end-to-end flows the rest of this document describes in prose and schematic pipelines. Arrows always flow downward; a colored arrow marks a **success continuation** or a **rejection/error exit**, grey marks a plain sequential or merge step.

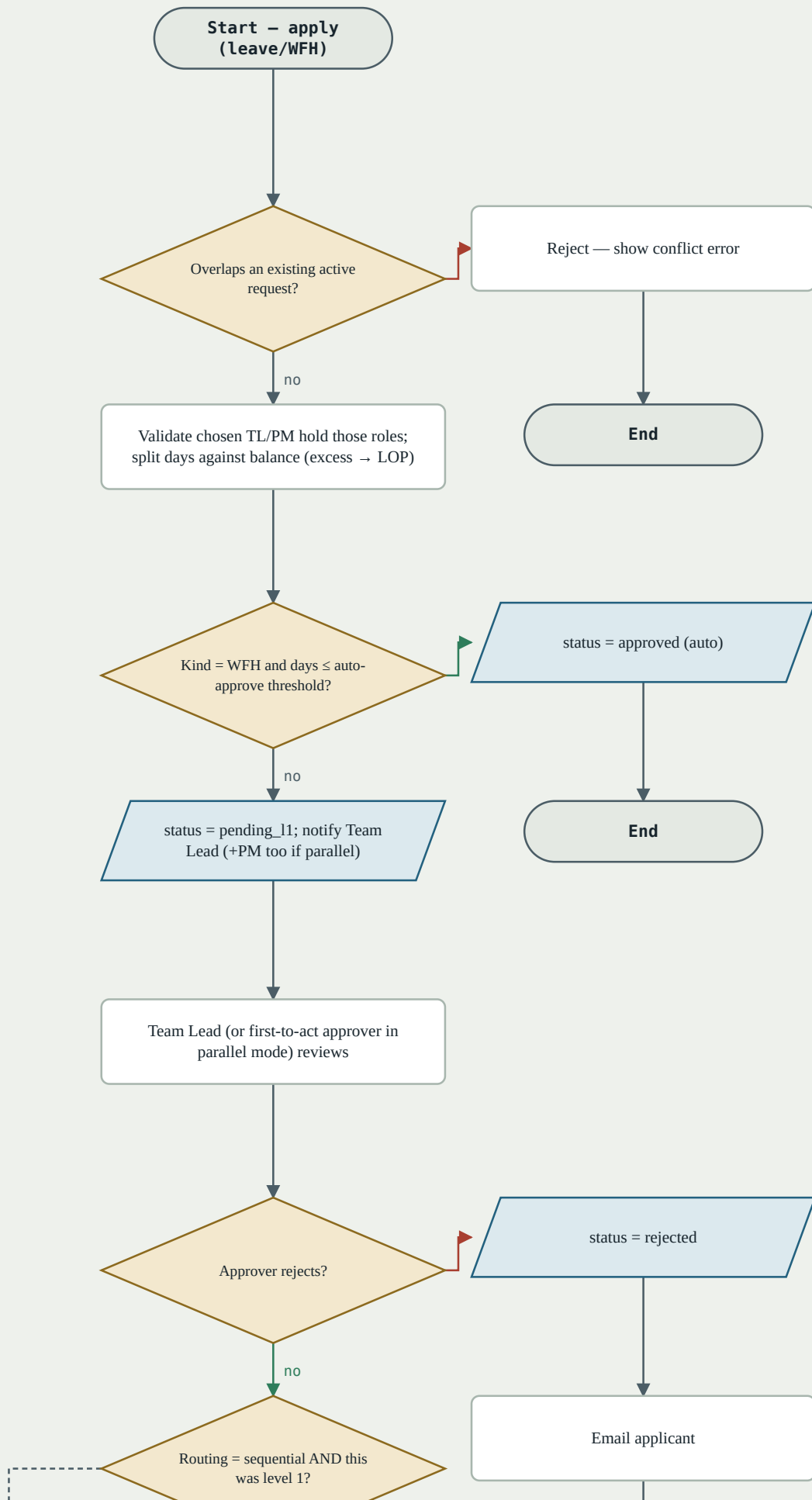


Benefit Wallet – submit to reimbursement





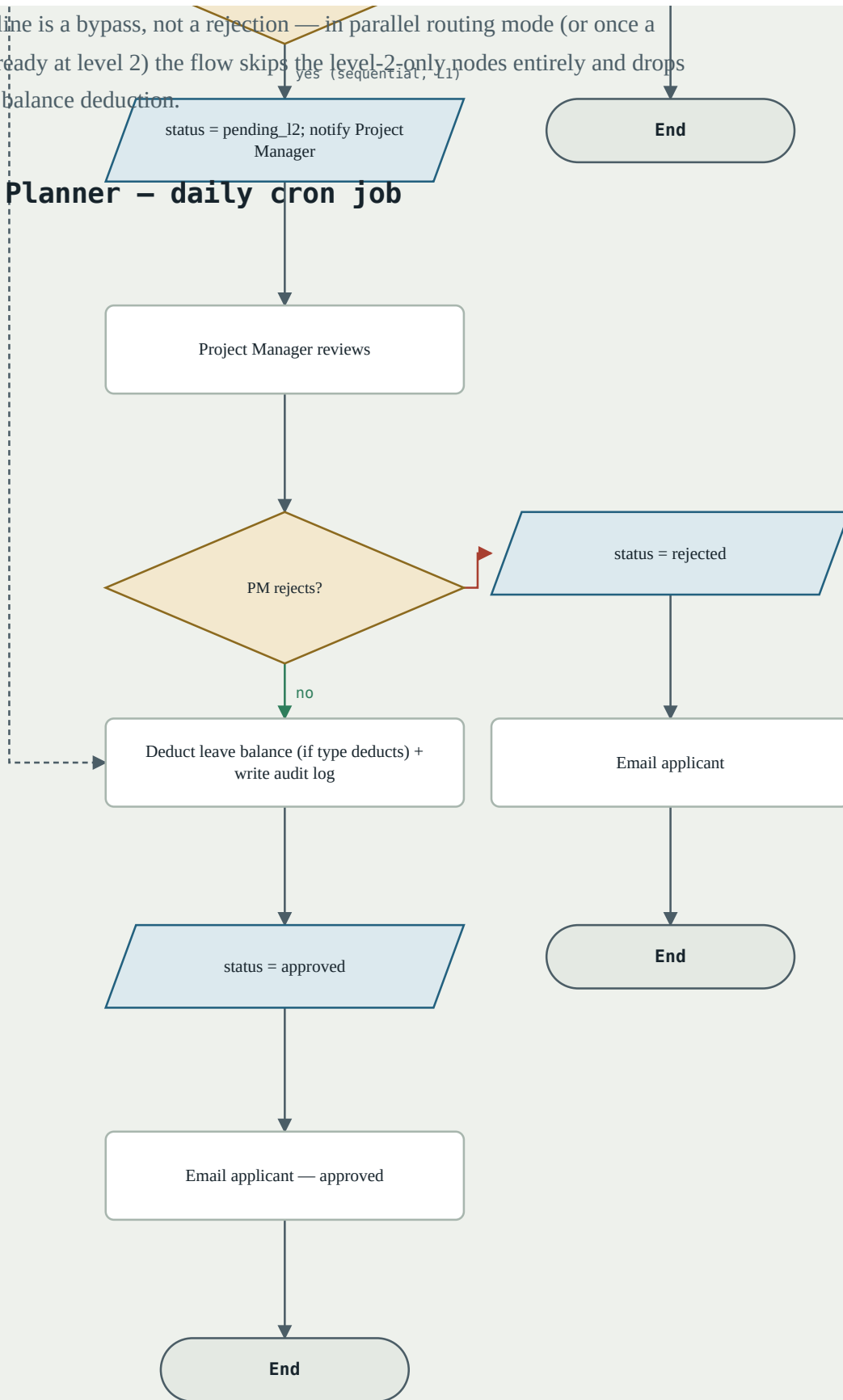
Leave & WFH – apply to balance deduction

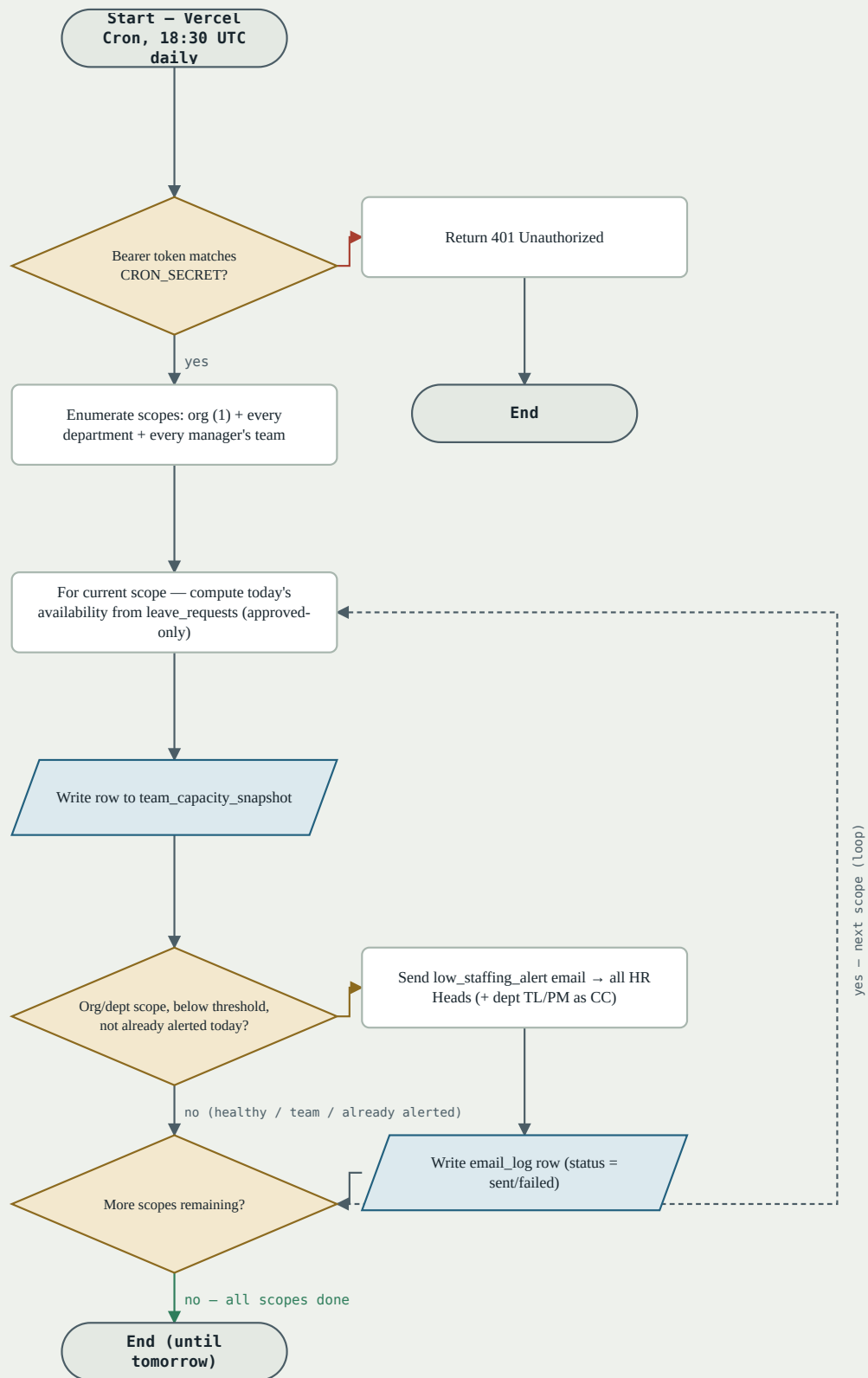


The dashed line is a bypass, not a rejection — in parallel routing mode (or once a request is already at level 2) the flow skips the level-2-only nodes entirely and drops straight into balance deduction.

Capacity Planner – daily cron job

no – parallel, skip to deduction





The dashed line is the one real loop in this document — the job repeats steps 3–5 once per scope (org, then each department, then each manager's team) before falling

through to **End** .

Verified against source on 2026-07-03 · file:line references were confirmed during research and omitted from this view for readability – ask for the citation list if you need to jump straight to code.

